

■ 4.1.2 Procedural Programming

Procedural programming is the standard approach used in traditional computer languages such C, Pascal, FORTRAN and BASIC. The basic idea is to have a program specify the sequence of steps that implements a particular algorithm.

```
BitCount[n_Integer] :=
  Block[ { ct = 0, m = n } ,
    While[ m != 0 ,
      If[ Mod[m, 2] != 0, ct++ ] ;
      m = Quotient[m, 2] ;
    ] ;
    Return[ ct ]
  ]
```

A procedural program to count the number of ones in the binary representation of an integer.

The simplest procedural programs consist of a sequence of commands which are executed in order. The commands typically set values for variables, which are then used later in the program. The “state” of the program at a particular point is determined by the values of the various variables that are in use.

You can think of the sequence of input lines in a particular *Mathematica* session as being like the steps in this kind of simple procedural program.

In the next level of procedural programming, one introduces the notion of *conditionals*. The *Mathematica* expression `If[predicate, branchT, branchF]` is a *conditional* which executes the commands *branch_T* if the predicate is `True`, and the commands *branch_F* if it is false. Conditionals are the simplest way to control the *flow* of a procedural program. When you get to a particular point in your program, you can explicitly test a given condition, and decide on the basis of it what commands to execute.

Sequences of commands in procedural programs form *blocks*. In *Mathematica*, the commands in a block are separated by semicolons. The *Mathematica* function `Block` lets you introduce variables that are “local” to a particular block.

Using blocks is important in producing structured programs. Each block can correspond to a “module” of your program, with a certain degree of independence from other modules.

Another important element of procedural programming is *iteration*. You can create loops that are executed repeatedly using *Mathematica* iteration functions like `While` and `For`. In the simplest cases, you know how many times a particular set of commands has to be executed. In more complicated cases, you can use iteration functions to execute a sequence of commands repeatedly, until, for example, a

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

particular condition stops being true. At each step in the iteration, you typically increment or otherwise update some particular set of variables that you are using.

Conditionals and iteration give you two ways to control the *flow* of your program. Conditionals let you choose to execute different branches in your code; iteration lets you loop over a single piece of code. Sometimes the flow of control in your programs may become sufficiently complicated that it can no longer be specified just by conditionals and iteration. Often this is a sign of inadequate structure in your program. Sometimes, however, it is inevitable. In such cases, you can use the *Mathematica* function `Goto` to “transfer control” to a `Label` at a specific point in your program.

Procedural programming is the traditional approach to computer programming, embodied in the traditional programming languages. It has had the historical advantage of being easy to implement on standard computer hardware.

Most of the computer algorithms you find now written out in books are given as procedural programs, often in a pseudo-Pascal language. The procedural programming functions in *Mathematica* allow you to implement these kinds of programs almost directly in *Mathematica*.

There are nevertheless many kinds of problems that are ill-suited to procedural programming. The constraints of traditional programming languages have often led people to formulate even these problems as procedural programs. But with a language like *Mathematica*, you have a wider range of choices.

If you are used to a procedural programming language like FORTRAN, you will often find that you instinctively try to create *Mathematica* programs that look like FORTRAN. It is certainly possible to use *Mathematica* in this way, but you will not get the best out of it.

Procedural programming is in many respects the lowest-level form of programming that *Mathematica* supports. When you write a procedural program, you have to specify explicitly many details about the exact flow of control, and so on. When you use higher-level forms of programming, you do not see these kinds of details, and it is easier for you to concentrate on the main issues in your program.

In fact, you will often find that the best way to create a *Mathematica* program is to start from the original mathematical description of your problem, and try to implement that as directly as possible. Thinking about your problems in terms of procedural programs is often a handicap in creating the most efficient and clear *Mathematica* programs.